RESOLVER LA ACTIVIDAD QUE SE PLANTEA EN EL DOCUMENTO ACTIVIDAD_5.4

1. **Una práctica entregada exactamente igual que la corrección del profesor, se evaluará con una nota de 6.**
2. **Se evaluará la práctica expresada en las propias palabras del alumno.**
3. **Se evaluará la corrección ortográfica y que las frases estén bien construidas. En general se evaluará la legibilidad del documento.**
4. **Se evaluará la presentación del documento.**
5. **Se penalizarán dos prácticas iguales, tanto en presentación, como en contenido.**
6. **SE PODRÁ SOLICITAR QUE EL ALUMNO/A DEFIENDA SU TRABAJO EN PÚBLICO**

RÚBRICA DE EVALUACIÓN

0 – 6 Puntos: la solución al problema de la práctica. 0 puntos no está resuelta. 6 puntos la práctica está resuelta correctamente.

0 – 0.5 Puntos: el formato del fichero ha de ser PDF. 0 puntos el formato del fichero no es PDF. 1 puntos el formato del fichero es PDF.

0 – 1 Puntos: formato de presentación. 0 la presentación es mala. 1.5 la presentación es muy buena y aporta referencias bibliográficas adecuadamente.

0 – 1.5 Puntos: expresión y corrección lingüística. 0 no se expresa adecuadamente y comete faltas de ortografía. 1.5 se expresa correctamente y con sus propias palabras y no comete faltas de ortografía.

0 – 1 Puntos. La presentación del documento es excelente, y además de expresarse correctamente, con sus propias palabras y no comete faltas de ortografía, aporta información adicional relevante con el tema del ejercicio, sumando valor al conjunto total del documento.

Enunciado de la Actividad:

ACTIVIDAD 5.4

Desarrolla esta actividad apoyándote en el documento de teoría. El punto 5 del documento de teoría **"Programa de ejemplo para persistencia de objetos con Hibernate y SpringBoot",** sigue los pasos que se indican para ***"Crear un programa de ejemplo que crea una sede, un departamento de esa sede y un empleado de ese departamento"*** en el IDE **IntelliJ Idea Community**

Edition. Genera un documento que refleje los pasos dados con comentarios e imágenes de los comandos y resultados de los scripts.

Sube el código desarrollado a tu perfil en GitHub y adjunta el enlace en el documento anterior.

Respuesta:

En esta actividad se ha desarrollado un ejemplo de persistencia de objetos con Hibernate y Spring Boot,

En primer lugar, se han creado las entidades Java Sede, Departamento y Empleado, estableciendo la correspondencia entre clases y tablas de la base de datos proyecto_orm.

Después se han creado los repositorios correspondientes mediante JpaRepository, lo que permite realizar operaciones de persistencia sin escribir consultas SQL manuales.

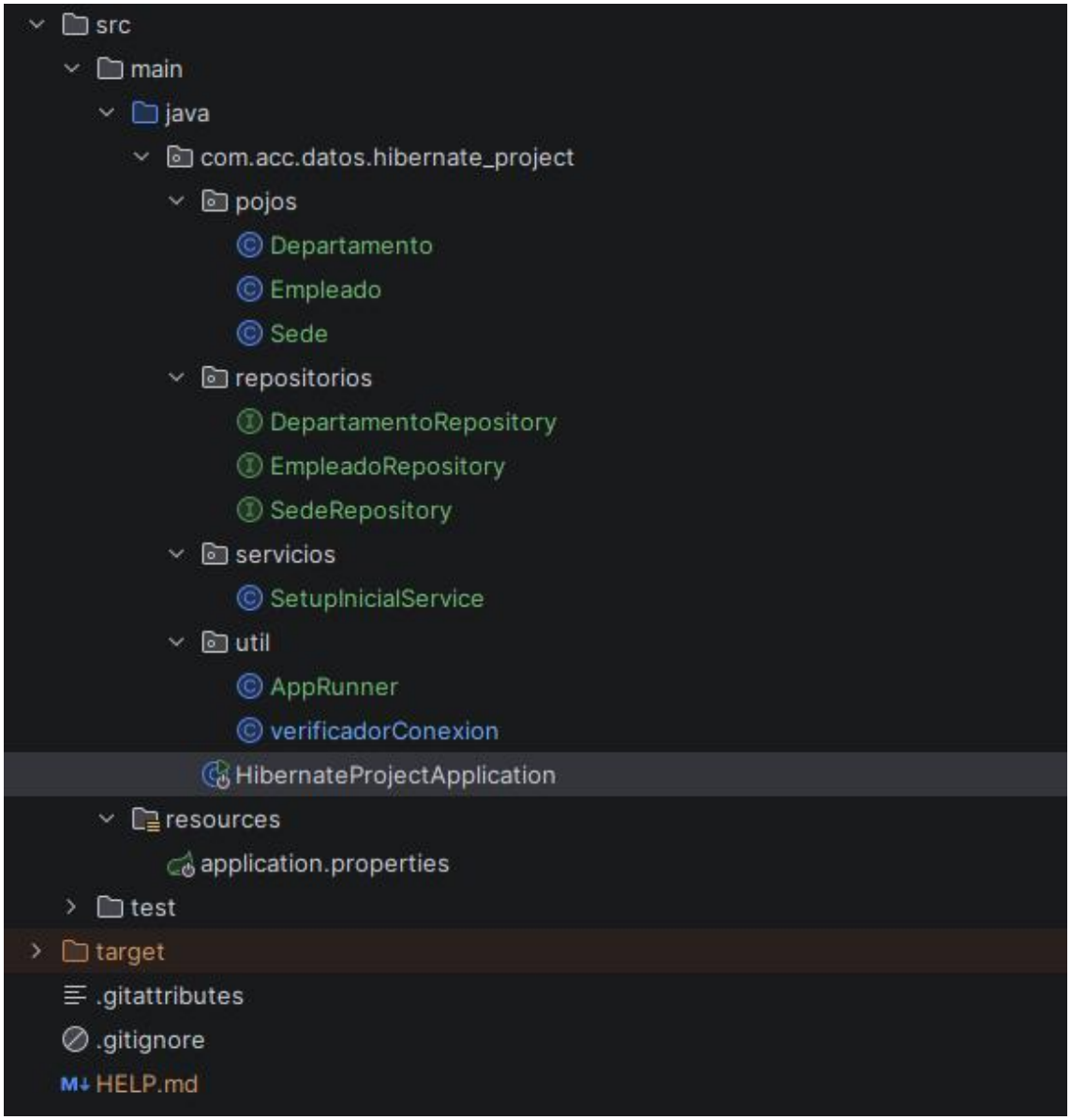
A continuación, se ha implementado un servicio llamado SetupInicialService, anotado con @Service, que contiene la lógica para crear una sede, un departamento asociado a dicha sede y un empleado asociado a ese departamento.

Este proceso se ha realizado dentro de una transacción mediante la anotación @Transactional, de forma que si ocurre un error, Spring deshace automáticamente los cambios.

Finalmente, mediante una clase AppRunner que implementa CommandLineRunner, se ha ejecutado esta lógica al arrancar la aplicación. Se ha comprobado en la consola que Hibernate genera las sentencias SQL de inserción y, posteriormente, se ha verificado en MySQL Workbench que los registros se han creado correctamente y que las claves foráneas apuntan a los valores esperados.

Capturas:

Estructura de clases creadas:



Estado de las tablas modificadas:
Sede:

```
1 •  SELECT * FROM sede;
2    SELECT * FROM departamento;
3 •  SELECT * FROM empleado;
```

Result Grid | Filter Rows: | Edit: | Export/Imp

id_sede	nom_sede
1	Madrid Central
NULL	NULL

Departamentos:

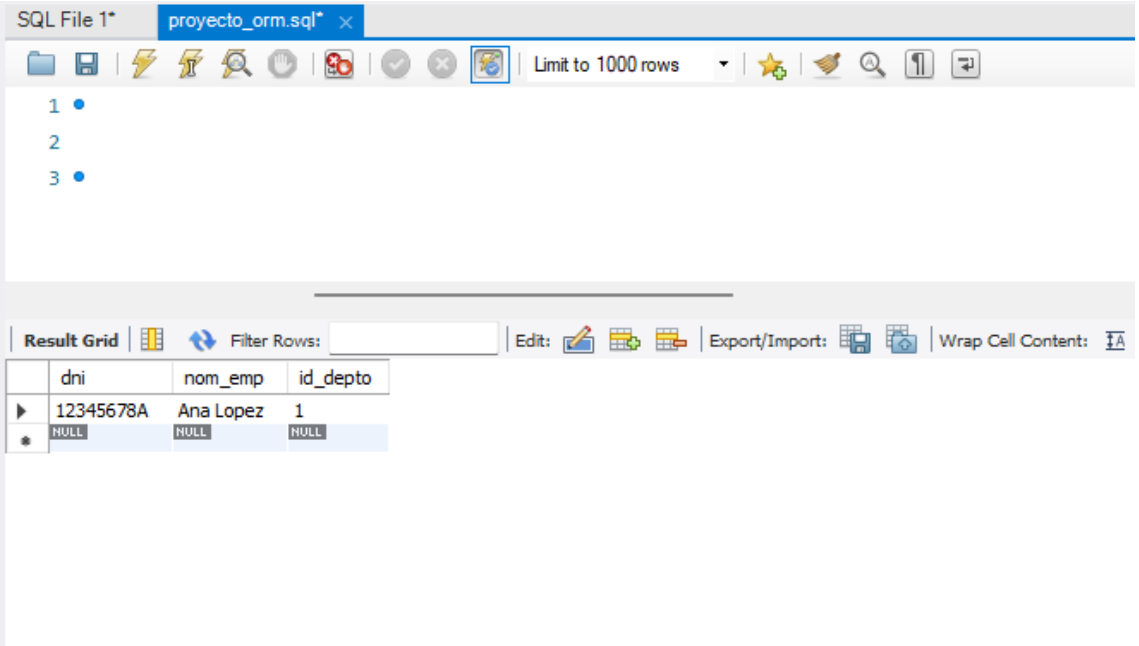
SQL File 1* | proyecto_orm.sql* | Limit to 1000 rows

```
1 •
2
3 •
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_depto	nom_depto	id_sede
1	Tecnologias de la Informacion	1
NULL	NULL	NULL

Empleado:



Resultados de la ejecución:

```
=======================================================
APLICACIÓN ARRANCADA. EJECUTANDO LÓGICA INICIAL...
=======================================================
--- CREANDO DATOS INICIALES DE FORMA EXPLÍCITA ---
Hibernate:
    insert
    into
        sede
        (nom_sede)
    values
        (?)
Sede creada con ID: 1
Hibernate:
    insert
    into
        departamento
        (nom_depto, id_sede)
    values
        (?, ?)
Departamento creado con ID: 1
Hibernate:
    select
        e1_0.dni,
        e1_0.id_depto,
        d1_0.id_depto,
        d1_0.nom_depto,
        d1_0.id_sede,
        s1_0.id_sede,
        s1_0.nom_sede,
        e1_0.nom_emp
    from
```

```
        empleado e1_0
    join
        departamento d1_0
            on d1_0.id_depto=e1_0.id_depto
    join
        sede s1_0
            on s1_0.id_sede=d1_0.id_sede
    where
        e1_0.dni=?
Empleado creado con DNI: 12345678A
--- DATOS INICIALES CREADOS CON ÉXITO ---
Hibernate:
    insert
    into
        empleado
        (id_depto, nom_emp, dni)
    values
        (?, ?, ?)
=========================================================
LÓGICA INICIAL FINALIZADA. La aplicación está lista.
=========================================================
```

Link a GitHub:

https://github.com/DGuelar/Practicas/tree/main/T5/hibernate-project/src/main/java/com/acc/datos/hibernate_project